GRCPC 2025 Editorial

A. zerorush

Author: Anastasis Koliopoulos.
Analysis: Anastasis Koliopoulos.

Tags: Greedy.

To begin, we reduce the problem to a decrease-only scenario by considering the absolute values of the sequence. Next, we observe that all operations must be applied to a single element — specifically, the smallest even number. This choice is optimal, since it will reach zero in the fewest steps among all even numbers. Note that an odd number can never become zero, so a sequence consisting entirely of odd numbers admits no solution. Let a denote the selected number, and let k be the number of operations needed for a to reach zero. Then

$$a - 2k = 0 \iff a = 2k \iff k = \frac{a}{2},$$

which is the answer to the problem.

The above approach requires only a single pass through the sequence, yielding a time complexity of $\mathcal{O}(N)$.

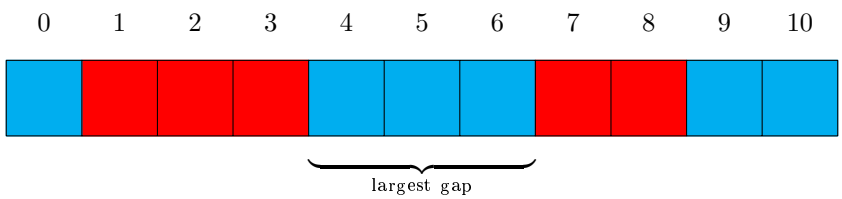
Indicative code

B. foodbreak

Author: Anastasis Koliopoulos.
Analysis: Anastasis Koliopoulos.

Tags: Brute force, Sortings.

Let us first consider a simple, non-optimal but intuitive approach. We define an array $A[0 \dots T]$, where $A[i]$ = red if there is a presentation at timestamp i, and $A[i]$ = blue otherwise. For each presentation $i \in [n]$, we set every cell in the segment $[s_i, e_i)$ to red. After processing all presentations, we find the longest contiguous segment consisting of cells with blue color, which gives us the answer.



This algorithm runs in worst-case time $\mathcal{O}(N \cdot \max_{1 \le i \le n}\{e_i - s_i\} + T) = \mathcal{O}(N \cdot T + T) = \mathcal{O}(N \cdot T)$, and requires $\mathcal{O}(T)$ space, which is insufficient to meet the time and memory limits. Let us now consider a better approach.

C. mansion

Author: Anastasis Koliopoulos.
Analysis: Anastasis Koliopoulos.

Tags: Graphs, SSSP, Dijkstra's algorithm, 0/1 BFS.

We model the problem as follows. Let $G = (V, E)$ be a directed graph with $|V| = N$ and $|E| = M$, where each edge $e \in E$ has weight $w(e) \in \{0, 1\}$. The goal is to find the shortest path from vertex 1 to vertex N, or determine that no such path exists.

Since all edge weights are non-negative, we can apply *Dijkstra's algorithm* to compute the shortest path from vertex 1 to vertex N. Implementing Dijkstra's algorithm using an *std::set* or an *std::priority_queue* yields a time complexity of $\mathcal{O}((N + M) \cdot \log N)$, which is sufficient to pass all the test cases. Using a *Fibonacci heap* we can reduce the time complexity to $\mathcal{O}(N \cdot \log N + M)$, but its complex implementation and its large constant factor makes it impractical for competitive programming contests.

Actually, since all edge weights are either 0 or 1, we can instead use the more efficient *0/1 BFS* algorithm, hich avoids the logarithmic factor and runs in $\mathcal{O}(N + M)$ time.

Indicative code

D. packages

Author: Anastasis Koliopoulos.
Analysis: Anastasis Koliopoulos.

Tags: Binary search, k-th element.

We define the function $c : \{0, 1, \ldots, n\} \to \mathbb{N}$, where $c(k)$ denotes the minimum cost to deliver k packages. We make the following crucial observation.

Observation 1. The cost function is non-decreasing.

Proof. We'll show that $c(k) \leq c(k+1)$, where $0 \leq k \leq n-1$. We denote by I_{k+1} the set of indices of the packages selected in an optimal solution with $k+1$ packages. For every $i \in I_{k+1}$ we have

$$\begin{aligned} k < k+1 &\implies k \cdot B[i] \leq (k+1) \cdot B[i] && (B[i] \geq 0) \\ &\implies A[i] + k \cdot B[i] \leq A[i] + (k+1) \cdot B[i]. \end{aligned}$$

Writing the above inequality for every i, we get

$$\left.\begin{aligned} A[i_1] + k \cdot B[i_1] &\leq A[i_1] + (k+1) \cdot B[i_1] \\ A[i_2] + k \cdot B[i_2] &\leq A[i_2] + (k+1) \cdot B[i_2] \\ &\vdots \\ A[i_{k+1}] + k \cdot B[i_{k+1}] &\leq A[i_{k+1}] + (k+1) \cdot B[i_{k+1}] \end{aligned}\right\} \overset{+}{\implies} \sum_{i \in I_{k+1}} A[i] + k \cdot B[i] \leq c(k+1).$$

Subtracting $A[j] + k \cdot B[j]$ from the left-hand side, for some $j \in I_{k+1}$, we obtain

$$\underbrace{\sum_{i \in I_{k+1} \setminus \{j\}} A[i] + k \cdot B[i]}_{\text{a feasible } k\text{-packages solution}} \geq c(k).$$

Combining the two inequalities above, we get

$$c(k) \leq c(k+1),$$

which completes the proof.

■

Leveraging the monotonicity of the cost function, we can *binary search the answer*. We choose a value $k \in \{0, 1, \ldots, n\}$ and compute $c(k)$. If $c(k) \leq X$, where X denotes the available budget, we continue searching among values $k' \geq k$; otherwise, we restrict the search to values $k' < k$.

To determine which packages to deliver for a fixed k, we create the array $C[1 \dots n]$, where

$$C[i] = A[i] + k \cdot B[i]$$

and retrieve the k cheapest packages. There are three ways to obtain the desired packages, described below:

- Sort the array C in $\mathcal{O}(N \cdot \log N)$ time.
- Use the built-in C++ function *nth_element*, which runs in $\mathcal{O}(N)$ time on average.
- Use the *Median of medians algorithm*, which guarantees $\mathcal{O}(N)$ time in the worst case.

The binary search contributes an additional $\mathcal{O}(\log N)$ factor.

Overall, the best achievable time complexity is $\mathcal{O}(N \cdot \log N)$ in the worst case. Nevertheless, all of the above approaches are efficient enough to pass the given constraints. In the provided implementation, we use the second approach.

Indicative code

E. javelins

Author: Anastasis Koliopoulos.
Analysis: Anastasis Koliopoulos.

Tags: Exchange arguments, DP, State rotation, Space optimization.

The only difference from the previous problem is the cost function. If we select

F. audit

Author: Anastasis Koliopoulos.
Analysis: Anastasis Koliopoulos.

Tags: segment tree, coordinate compression, offline queries